



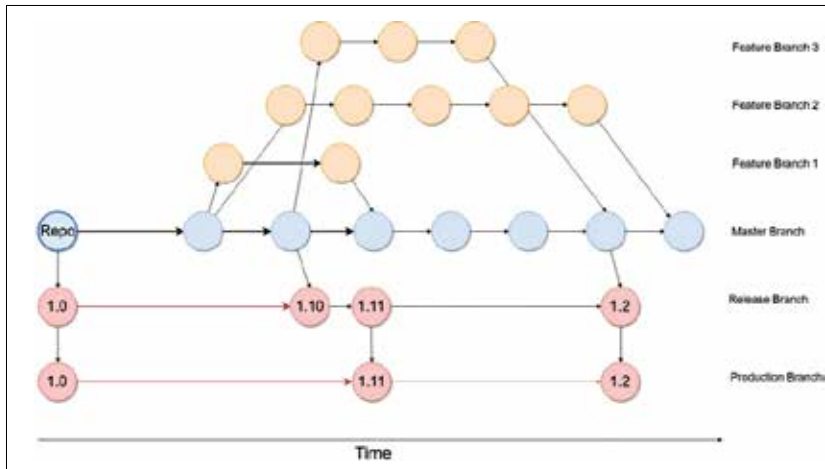
Free GitHub tools

GitHub has made all of its core tools free for developer teams, which greatly helps those working from home. <https://dgit.in/may20-13>



Deepfake ads

Advertising agencies are recycling old content by using a number of technologies, including deepfake. <https://dgit.in/may20-14>



Having too many feature flags can complicate things

the other modules are maintained in the section maintained by the vendor. This way Google can continuously push new security updates to all devices in the Android ecosystem without having to rely on the vendor to push updates.

What we're trying to say is that you should avoid fragmentation hell i.e. your code ends up with so many decision trees that even you have no idea as to what kind of impact each and every toggle/flag can have. This brings us to the next step in incorporating feature flags – monitoring.

MONITORING AND CLEAN-UP

If you don't need it, remove it. It doesn't get simpler than this. When

your application reaches the production stage, it will be pretty massive with lots of components, modules, databases and even server instances. At such a scale, each feature flag that you introduce into the application creates a separate branch that requires a certain amount of hardware resources. Also, you might end up affecting more than one feature in a cascading fashion because of the new feature flag that you've introduced. Now if you have multiple feature flags which have such cascading effects, then the complexity of fragmentation goes up by a large magnitude. Also, there is a chance that you might

forget that you had introduced a new flag a couple of months ago which has not been toggled in ages. In such a case, you have a code branch that's doing absolutely nothing but eating up resources. Ideally, you'd want to deprecate that code branch and remove that useless flag.

This is where monitoring comes into the picture. You should monitor and constantly observe all your feature flags to gauge whether you actually need them. If you're not going to use a feature flag, you should do away with it immediately. In smaller projects, you can maintain a simple list of all the feature flags but when the scale and scope of the project goes up, you might want to explore using independent solutions to perform this task for you. There are tools such as Sonar and CodeClimate which allow



LaunchDarkly is a popular Feature Flag platform

you to constantly test your code to see functionality and possibly generate heatmaps of which functions or modules are in use. These software will let you know what's not being used and then you can simply remove the feature flag from the code.

POPULAR FEATURE FLAG PLATFORMS

Aside from testing and optimisation solutions such as Sonar and CodeClimate, there are specialised tools which help you maintain feature flags. There are solutions such as Optimizely, Split, ProbeBeta and LaunchDarkly which can perform this exact function for you. Depending on the scale of your website you can pick the appropriate plan for these services. LaunchDarkly starts at \$75/month for starter applications and as you scale up you can explore the next tier or even scale up to a customised enterprise plan for your application. [4](#)



Monitor your code to keep it clean of needless feature flags